

0.83.1. La fonction bruit() — un bruit

`#include <bruit>` : `bruit(duree, volume)` frappe sur la voix du bruit : un tir, un pas, une explosion.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `bruit(duree, volume)` frappe sur la **voix 4**, celle du bruit : pas une note, un choc. Un troisième argument, le **grain** (0 à 255), le rend plus sourd ou plus sifflant.

La ligne à écrire : `#include <bruit>`.

Ses arguments : la durée (1 à 64, ou 0 : sans fin), le volume (0 à 15), et le grain, facultatif.

Ce qu'elle coûte : environ **50 octets**.

Essaie : `bruit(20, 8, 0x55);` : plus rêche.

On retrouve bruit dans 6 leçons

[0.83.2](#), [0.103](#), [0.104](#), [53](#), [57](#), [122](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **451 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      61 o ← la console, à l'allumage – la mise en route de la console
main()         65 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   76 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
LireManette    37 o ← bouton() – lit les huit boutons
Tuiles        112 o ← CopierTuiles – 7 lettres sur 43
"APPUIE SUR A" 12 o ← texte() – un texte gravé
```

Lettres gravées : AEIPRSU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

```
bruit() × 1    39 o sur place, dans main()
texte() × 1    11 o sur place, dans main() + EcrireTexte (10 o)
bouton() × 1    5 o sur place, dans main() + LireManette (37 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

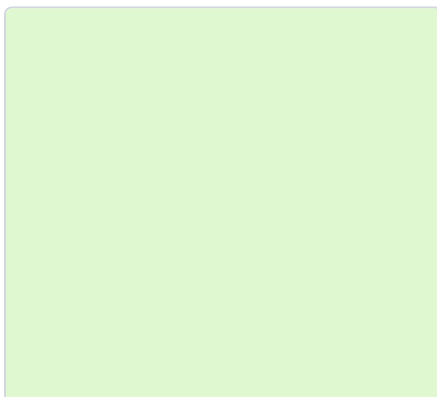
```
// ---- #include <bruit> : un bruit ----  
// Chaque fois qu'on appuie sur A, un bruit court.  
  
#include <bruit>      // bruit() : la voix du bruit  
#include <bouton>  
#include <texte>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(4, 8, "APPUIE SUR A");  
  
    while (true) {  
        image();  
        if (bouton(A)) {  
            bruit(10, 12);    // court (10), fort (12)  
        }  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.

APPUIE SUR A

A — l'écran n'a pas bougé

5. Ce qu'on doit voir

APPUIE SUR A ; le bouton A fait un bruit sec.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ silence au départ
- ✓ A : la voix du bruit joue

Ensuite : **0.83.2. Un son quand on ramasse : note — et un bruit contre le mur** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++